

Lab 03 RBAC and privileged access

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a access matrix with a least-privilege change proposal. This lab supports LO3. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `SCOPED`, `REVIEW_BROAD`, `SCOPED`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

RBAC scope inheritance

Permissions at parent scope apply to child resources; additional Reader access does not reduce Contributor.

Contract: principalId, roleDefinitionId, scope

Accepted case: Reader at rg-sec provides read-only management access.

Counterexample: Contributor at subscription remains effective below rg-sec.

Diagnosis: Inspect all applicable assignments and deny assignments before changing local roles.

Evidence: Preserve principal ID, full scope and inherited role list.

Custom role actions

Management actions and data actions authorise different operations; subtracting NotActions is not a global deny.

Contract: Actions, NotActions, DataActions, AssignableScopes

Accepted case: A custom inventory role grants Microsoft.Resources/subscriptions/resourceGroups/read.

Counterexample: Granting Microsoft.Storage/storageAccounts/listKeys/action exposes account keys.

Diagnosis: Remove unnecessary key-list access and review other assignments that may still grant it.

Evidence: Compare allowed action strings against the approved job duties.

Privileged activation

An eligible privileged role becomes active for an approved bounded interval.

Contract: principal, role, justification, start_utc, end_utc

Accepted case: An approved 30-minute activation expires at 10:30Z.

Counterexample: A standing Owner assignment has no expiry.

Diagnosis: Revoke unnecessary standing access; verify emergency access separately from ordinary roles.

Evidence: Record eligible versus active state, approver and actual expiry.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Add Reader to u-owner. The result must remain REVIEW_BROAD because a narrower grant does not remove Owner.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure A Establish the sandbox and role evidence

1. Sign in to Azure Portal. Use the directory/subscription selector to choose the trainer-approved subscription. Record its name and ID; stop if the selected subscription is not the training allocation.
2. Open Resource groups > Create. Select the assigned subscription and region. Name the group rg-itsec-<initials> and create it. Record the resulting full resource-group ID.
3. Open the new group > Access control (IAM) > Check access. Select the approved test principal. Record every direct and inherited role, including the scope of parent assignments.
4. With Role Based Access Control Administrator or another authorised role-assignment permission, choose Add role assignment, select Reader, choose the test principal and use the resource-group scope. Review and assign. If unavailable, record the exact denied operation and request an authorised trainer demonstration.
5. Confirm the assignment appears. Using the approved test principal, verify resource inspection succeeds and an ungranted write operation is denied. A principal with inherited Contributor cannot serve as a valid negative-test subject; explain why and select the correct sandbox principal with the trainer.
6. Open IAM > Add > Add custom role if authorised. Name it ITSEC Inventory Reader <initials>. In Permissions, add only Microsoft.Resources/subscriptions/resourceGroups/read. Set Assignable scopes to the training resource group. Review the JSON before creation. Do not grant wildcard write or storage-key access. Record the role definition, assignment scope and its limited permitted action.
7. Compare the custom role with the legacy storage-key reader requirement. Locate Microsoft.Storage/storageAccounts/listKeys/action in a role definition and explain that access to account keys can enable broad data access. Demonstrate inspection of the action; add it only if the trainer explicitly allocates an isolated storage-key exercise. Do not capture key values.
8. Microsoft Entra admin center > Identity governance > Privileged Identity Management > Azure resources: select the training scope. Review eligible and active assignments. Where allocated, request a short approved activation with a justification. Record activation and expiry; otherwise record the licence/permission blocker and trainer-demonstrated evidence.
9. Save role JSON or screenshots, the test principal identifier, full scopes, actual allow/deny results and UTC times. Remove only the Reader/custom-role assignments created for this lab after the trainer confirms evidence is complete. Keep the resource group for later labs.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a access matrix with a least-privilege change proposal with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use `py -3` on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase `true` and `false` in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.